Дополнительные объекты

```sql
CREATE TABLE IF NOT EXISTS "AuditLog" (
    id BIGSERIAL PRIMARY KEY,
    table_name TEXT NOT NULL,
    operation TEXT NOT NULL,
    old_data JSONB,
    new_data JSONB,
    changed_by TEXT DEFAULT current_user,
    changed_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE IF NOT EXISTS "ErrorLog" (
    id BIGSERIAL PRIMARY KEY,
    error_code TEXT,
    error_message TEXT,
    error_detail TEXT,
    error_hint TEXT,
    context JSONB,
    occurred_at TIMESTAMPTZ DEFAULT now()
);
```

Функции

```sql
CREATE OR REPLACE FUNCTION fn_completed_courses(p_student_id INT)
RETURNS INT
LANGUAGE sql
STABLE
AS $$
    SELECT COUNT(*)
    FROM "Enrollment"
    WHERE student_id = p_student_id AND progress = 100.0;
$$;

CREATE OR REPLACE FUNCTION fn_avg_progress(p_student_id INT)
RETURNS NUMERIC(5,2)
LANGUAGE sql
STABLE
AS $$
    SELECT COALESCE(AVG(progress), 0.0)
    FROM "Enrollment"
    WHERE student_id = p_student_id;
$$;

CREATE OR REPLACE FUNCTION fn_is_teacher(p_user_id INT)
RETURNS BOOLEAN
LANGUAGE sql
STABLE
AS $$
    SELECT role = 'teacher' FROM "User" WHERE id = p_user_id;
$$;
```

Хранимые процедуры

Процедура зачисления студента на курс

```
CREATE OR REPLACE PROCEDURE sp_enroll_student(
    p_student_id INT,
    p_course_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_user_role TEXT;
    v_course_exists INT;
BEGIN
    SELECT role INTO v_user_role FROM "User" WHERE id = p_student_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'User with ID % does not exist', p_student_id;
    END IF;
    IF v_user_role != 'student' THEN
        RAISE EXCEPTION 'User with ID % is not a student (role=%)',
           p_student_id, v_user_role;
    END IF;
    SELECT COUNT(*) INTO v_course_exists FROM "Course" WHERE id =
       p_course_id;
    IF v_course_exists = 0 THEN
        RAISE EXCEPTION 'Course with ID % does not exist', p_course_id;
    END IF;
    INSERT INTO "Enrollment" (student_id, course_id, progress)
    VALUES (p_student_id, p_course_id, 0.0);
    RAISE NOTICE 'Student % successfully enrolled in course %',
       p_student_id, p_course_id;
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'Error: student % is already enrolled in course %'
           , p_student_id, p_course_id;
        INSERT INTO "ErrorLog" (error_code, error_message, context)
        VALUES (SQLSTATE, SQLERRM, jsonb_build_object('student_id',
           p_student_id, 'course_id', p_course_id));
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'Foreign key error: please check if student or
           course exists';
        INSERT INTO "ErrorLog" (error_code, error_message, context)
        VALUES (SQLSTATE, SQLERRM, jsonb_build_object('student_id',
           p_student_id, 'course_id', p_course_id));
    WHEN OTHERS THEN
        RAISE NOTICE 'Unknown error: %', SQLERRM;
        INSERT INTO "ErrorLog" (error_code, error_message, context)
        VALUES (SQLSTATE, SQLERRM, jsonb_build_object('student_id',
           p_student_id, 'course_id', p_course_id));
END;
$$;
```

Процедура отчисления студента с курса

```
CREATE OR REPLACE PROCEDURE sp_unenroll_student(
    p_student_id INT,
    p_course_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_enrollment_id INT;
BEGIN
    SELECT id INTO v_enrollment_id
    FROM "Enrollment"
    WHERE student_id = p_student_id AND course_id = p_course_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Student % is not enrolled in course %',
           p_student_id, p_course_id;
    END IF;
    DELETE FROM "Enrollment" WHERE id = v_enrollment_id;
    RAISE NOTICE 'Student % unenrolled from course %', p_student_id,
       p_course_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error during unenrollment: %', SQLERRM;
        INSERT INTO "ErrorLog" (error_code, error_message, context)
        VALUES (SQLSTATE, SQLERRM, jsonb_build_object('student_id',
           p_student_id, 'course_id', p_course_id));
END;
$$;
```

Триггеры

Триггер проверки прогресса

```
CREATE OR REPLACE FUNCTION trg_enrollment_progress_check()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF NEW.progress < OLD.progress THEN
            RAISE EXCEPTION 'Trigger: progress cannot decrease (was %,
               became %)', OLD.progress, NEW.progress;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;
DROP TRIGGER IF EXISTS t_enrollment_progress_check ON "Enrollment";
CREATE TRIGGER t_enrollment_progress_check
BEFORE UPDATE ON "Enrollment"
FOR EACH ROW
EXECUTE FUNCTION trg_enrollment_progress_check();
```

Триггер автоматической установки даты зачисления

```
CREATE OR REPLACE FUNCTION trg_enrollment_set_enrolled_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.enrolled_at IS NULL THEN
        NEW.enrolled_at = CURRENT_TIMESTAMP;
    END IF;
    RETURN NEW;
END;
$$;
DROP TRIGGER IF EXISTS t_enrollment_set_enrolled_at ON "Enrollment";
CREATE TRIGGER t_enrollment_set_enrolled_at
BEFORE INSERT ON "Enrollment"
FOR EACH ROW
EXECUTE FUNCTION trg_enrollment_set_enrolled_at();
```

Примеры использования

```
-- View existing students and courses
SELECT id, name, email, role FROM "User" WHERE role = 'student';
SELECT id, title FROM "Course";

-- Enroll student in course
CALL sp_enroll_student(3, 2);
-- Try to enroll again (error)
CALL sp_enroll_student(3, 2);

-- Check enrollment was created
SELECT * FROM "Enrollment" WHERE student_id = 3;

-- Unenroll student from course
CALL sp_unenroll_student(3, 2);
-- Try to unenroll again (error)
CALL sp_unenroll_student(3, 2);

-- Use functions to get student statistics
SELECT
    id, name,
    fn_completed_courses(id) AS completed_courses,
    fn_avg_progress(id) AS avg_progress
FROM "User"
WHERE role = 'student';

-- Check teacher role
SELECT id, name, fn_is_teacher(id) AS is_teacher FROM "User";

-- View audit log
SELECT * FROM "AuditLog" ORDER BY changed_at DESC LIMIT 10;
-- View error log
SELECT * FROM "ErrorLog" ORDER BY occurred_at DESC;
```